Documento de Arquitetura e Requisitos
MVP: Sistema de Ponto Assíncrono

Versão 7

1. Visão Arquitetural e Decisões Técnicas

Este documento define a base técnica para o MVP do Sistema de Ponto. A arquitetura foi desenhada com foco em Alta Disponibilidade (AP), tolerância a falhas e interoperabilidade, adotando o modelo de Arquitetura Orientada a Eventos (EDA).

Decisões Core:

- **Agnóstico de Stack (Interoperabilidade):** A comunicação entre os serviços é feita de forma neutra, permitindo que produtores e consumidores sejam escritos em qualquer linguagem de programação (Java, Python, Go, Node.js, etc.).
- **Broker de Mensageria:** ActiveMQ.
- **Protocolo:** AMQP 1.0. A escolha do AMQP previne vendor lock-in e estabelece a fila de mensagens como um contrato agnóstico (API Assíncrona).
- **Padrão de Comunicação:** Coreografia de Eventos (Fila 1 para persistência -> Fila 2 para notificação).

2. Requisitos Funcionais (RF)

- **RF1 - Recepcionar Batida de Ponto (Producer):** A API deve receber a matrícula (ID) do funcionário, gerar o timestamp exato e enviar a mensagem para a Fila 1 (Processamento). Retorna 202 Accepted imediatamente.
- **RF2 - Persistir Dados (Consumer A):** Um worker lê a Fila 1, valida a mensagem e salva o registro no banco de dados (ex: H2/PostgreSQL).
- **RF3 - Acionar Notificação (Produtor Interno):** Após persistir com sucesso, o Consumer A emite um evento para a Fila 2 (Notificação).
- **RF4 - Comprovar Ponto (Consumer B):** Um worker lê a Fila 2 e simula o envio de e-mail (log) comprovando a operação.

3. Requisitos Não Funcionais (RNF)

- **RNF1 - Consistência Eventual:** Tolerância a atrasos de processamento na fila para atualização da leitura.

- **RNF2 - Garantia de Entrega:** Uso de Manual ACK (Acknowledge). A mensagem só é removida da fila após processamento bem-sucedido.
- **RNF3 - Tolerância a Falhas:** Se o banco falhar, a mensagem retorna para a Fila 1 (reprocessamento).

4. Regras de Negócio (RN)

- **RN1 - A Verdade é o Timestamp da API:** O horário oficial do ponto é o momento da requisição na API, não o da persistência no banco.
- **RN2 - Validação Tardia (Lazy Validation):** A API não consulta o banco para validar o ID. O Consumer A faz isso de forma assíncrona.
- **RN3 - Debounce (Anti-Duplicidade):** O sistema ignora batidas de ponto do mesmo ID num intervalo menor que 60 segundos.
- **RN4 - Comprovação Estrita:** A notificação de sucesso (Fila 2) só ocorre se o dado foi realmente persistido.

5. Design de Contratos (Payloads JSON)

Para garantir a independência de tecnologia, os contratos (JSONs) trafegados no ActiveMQ via AMQP são autodescritivos, utilizando o padrão ISO 8601 para datas e o conceito de Event Enrichment (Enriquecimento de Eventos).

5.1. Payload da API Pública (POST /pontos)

```
{
  "idFuncionario": "98765"
}
```

5.2. Contrato da Fila 1 (RegistrarPontoCommand)

```
{
  "mensagemId": "a1b2c3d4...",
  "idFuncionario": "98765",
  "timestampRegistro": "2026-10-14T20:38:00Z"
}
```

5.3. Contrato da Fila 2 (PontoRegistradoEvent)

```
{
  "mensagemId": "f9e8d7c6...",
  "idFuncionario": "98765",
  "nomeFuncionario": "João da Silva",
  "emailFuncionario": "joao@empresa.com",
  "timestampRegistro": "2026-10-14T20:38:00Z"
}
```

6. Estrutura do Repositório (Monorepo)

O projeto adota Monorepo para centralizar contratos e facilitar a execução local. A infraestrutura (Docker) fica na pasta /infra, isolada das pastas de cada aplicação.

```
sistema-ponto/
├── docs/                          # Documentação (DOCX/PDF de arquitetura)
├── infra/                         # docker-compose.yml (ActiveMQ, PostgreSQL) +
db/init (schema/seed)
└── apps/
    ├── apiProducer/               # Python 3.12/FastAPI (Recebe requisição e
enfileira na Fila 1)
    ├── persistenceConsumer/       # Java 21/Spring Boot (Lê Fila 1, Salva BD, Envia
Fila 2)
    └── notificationConsumer/      # Python 3.12/Qpid Proton (Lê Fila 2 e notifica)
```

7. Estratégia de Configuração e Ambientes (12-Factor App)

Configurações que variam entre ambientes são injetadas exclusivamente via Variáveis de Ambiente (ENVs), priorizando a Nuvem em PROD/QA e arquivos .env isolados na máquina local em DEV. A aplicação adota a política de "Fail Fast" para ausência de variáveis vitais em produção.

8. Modelagem de Dados (Relacional)

Para manter a simplicidade do ambiente open-source, o MVP utilizará um banco de dados relacional (H2 em memória para DEV, PostgreSQL via Docker para testes/produção).

Tabela: funcionario (Apenas Leitura no MVP)

```
CREATE TABLE funcionario (
    id VARCHAR(50) PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL
);
```

Tabela: registro_ponto (Append-only)

```
CREATE TABLE registro_ponto (
    id UUID PRIMARY KEY,
    mensagem_id UUID NOT NULL UNIQUE, -- ID da fila (Idempotência)
    id_funcionario VARCHAR(50) NOT NULL,
    timestamp_registro TIMESTAMP WITH TIME ZONE NOT NULL,
    criado_em TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_funcionario FOREIGN KEY (id_funcionario) REFERENCES
funcionario(id)
);
```

9. Topologia das Filas (Mensageria)

O fluxo principal utiliza duas filas no ActiveMQ. Para esta primeira iteração do MVP, o tratamento de falhas focará na rejeição natural (NACK) com retorno para a fila principal, sem a complexidade de Dead Letter Queues (DLQ).

- **Fila 1 (Persistência):** fila.ponto.processamento
  Propósito: Receber o comando da API. O Consumer A (Worker Persistence) consome desta fila.
- **Fila 2 (Notificação):** fila.ponto.notificacao
  Propósito: Receber o evento enriquecido do Consumer A. O Consumer B (Worker Notification) consome desta fila.

10. Trade-off do Teorema CAP e Ponto Único de Falha

10.1. Escolha: AP (Disponibilidade + Tolerância a Partição)

Quando há falha ou partição entre os componentes, o sistema prioriza a Disponibilidade em vez da Consistência: a batida de ponto é aceita e enfileirada mesmo que o banco de dados esteja fora ou atrasado. A API não consulta o banco (RN2) e a gravação acontece depois, de forma assíncrona (RNF1 - Consistência Eventual).

- **Ganho:** o funcionário consegue bater o ponto mesmo com o banco indisponível. Como o horário oficial é o da API (RN1), o atraso no processamento não altera o registro.
- **Custo - Leitura desatualizada:** por um intervalo, o ponto já foi aceito mas ainda não está gravado no banco.
- **Custo - Validação tardia:** funcionário inexistente e debounce de 60s só são verificados no Consumer A. A API pode responder 202 para uma batida que será descartada depois.
- **Custo - Entrega "pelo menos uma vez" (at-least-once):** em reprocessamentos, a gravação é protegida pela idempotência por mensagem_id, mas a notificação da Fila 2 pode ser enviada mais de uma vez.

10.2. Ponto Único de Falha (SPOF): o Broker

Hoje existe um único nó do ActiveMQ, e ele é o único caminho entre a API e os workers. Se o broker cair, a disponibilidade escolhida no CAP deixa de valer na entrada do sistema.

- **Broker fora:** a API responde 503 e a batida não fica guardada em lugar nenhum. O cliente precisa tentar de novo.
- **Broker lento:** cada requisição abre uma conexão nova com timeout de 10s, então a API pode demorar até 10s antes de responder 503 (o que também conflita com a resposta imediata do RF1).
- **Armazenamento do broker:** as mensagens são persistentes (durable), mas ficam em um único volume. Perder esse volume perde as batidas ainda não consumidas.

Mitigações previstas (fora do escopo do MVP):

- ActiveMQ em alta disponibilidade (primário/réplica com armazenamento compartilhado e failover configurado nos clientes).
- **Buffer local na API (padrão Outbox):** guardar a batida localmente e reenviar quando o broker voltar, mantendo o timestamp original.
- Retry com backoff no cliente que registra o ponto.

10.3. Pontos de Atenção

- **Debounce sob concorrência:** com mais de uma instância do Consumer A, duas batidas do mesmo funcionário podem passar ao mesmo tempo pela checagem de 60s. Com instância única isso não ocorre; ao escalar, a regra precisa de garantia no banco (lock ou constraint).
- **Mensagens com erro recorrente:** sem DLQ própria, uma mensagem que falha sempre com erro recuperável é reentregue até o limite da política de reentrega do ActiveMQ. Definir esse limite e o destino final.
- **Relógio da API:** o timestamp oficial vem do relógio da máquina da API. Com mais de uma réplica, os servidores precisam estar sincronizados (NTP).
- **Observabilidade:** monitorar a profundidade das filas e o tempo entre timestampRegistro e criado_em, que mede o atraso real da consistência eventual.